

Lab 4: Microservices Decomposition & Communication

System: BadNet - Badminton Social Network

Introduction

This lab focuses on transitioning from the monolithic layered architecture to a **Microservices Architecture**. The goal is to logically decompose the BadNet system into independent, loosely coupled services based on business capabilities, define clear service contracts, and establish communication strategies between services.

The microservices architecture will enable:

- Independent deployment and scaling
- Fault isolation
- Technology diversity
- Team autonomy
- Better maintainability for a growing badminton community platform

Activity Practice 1: Decomposition by Business Capability

1.1 Core Business Capabilities Identified

Based on the current codebase and functional features of BadNet, the following core business capabilities have been identified:

1. User Management
2. Profile & Social Feed
3. Challenge & Match Management
4. ELO Ranking System
5. Team & Club Management
6. Tournament Management
7. Virtual Betting System
8. Notification System

1.2 Proposed Microservices

Business Capability	Proposed Microservice	Data Owned (Main Entities)	Primary Responsibilities
User Management	User Service	Users, Profiles, Auth credentials	Registration, Login, Profile
Social Content	Social Service	Posts, Comments, Likes, Feeds	Create Post, Interaction
Challenge & Match	Match Service	Challenges, Matches, Results	Create/Accept/Record Match
Ranking & ELO	Ranking Service	ELO Ratings, Leaderboards, History	ELO Calculation, Ranking
Team Management	Team Service	Teams, Members	Create & Manage Teams
Tournament Management	Tournament Service	Tournaments, Registrations, Schedules	Create & Manage Tournaments
Virtual Betting	Betting Service	Bets, Virtual Coins, Transactions	Betting logic & Settlement
Notification	Notification Service	Notifications, Templates	Real-time & Email notifications

1.3 Service Boundaries & Coupling Analysis

- **Data Encapsulation:** Each service owns its own database.
- **Loose Coupling:** Services communicate only through APIs or events.
- **High Cohesion:** Related business logic is grouped together.

Key Dependencies:

- Match Service depends on User Service & Ranking Service
- Betting Service depends on Match Service
- Notification Service depends on multiple services (Match, Tournament, Betting)

Activity Practice 2: Defining Service Contracts

Focus: Match Service (Core feature of BadNet)

2.1 Match Service API Contract

Endpoint: Create Challenge

Property	Value
Endpoint	POST /api/challenges
Description	Create a new challenge
Authentication Required	(JWT)

Request Body:

JSON

```
{  
  "challenger_id": 5,  
  "opponent_id": 12,  
  "match_date": "2026-05-25T15:00:00",  
  "location": "District 7 Badminton Court",  
  "note": "Best of 3 sets"  
}
```

Response (201):

JSON

```
{  
  "success": true,  
  "message": "Challenge created successfully",  
  "data": { ... }  
}
```

Endpoint: Record Match Result

Property Value

Endpoint PUT /api/matches/{id}/result

Description Record final score and trigger ELO update

Other important endpoints: Get Challenge Details, Accept Challenge, Get User Challenges, etc.

Activity Practice 3: C4 Model (Level 1: System Context)

